

Gestion Scolaire - Documentation technique (v1.2.0)

DOCUMENTATION COMPLETE DU LOGICIEL DE GESTION SCOLAIRE SOMMAIRE

1. Vue d'ensemble
2. Arborescence complete du projet
3. Architecture et demarrage
4. Point d'entree : `main.py`
5. Configuration : `core/config.py`
6. Couche donnees : `database/db.py`
7. Depots : `repositories/`
8. Services et API : `services/` + `api/`
9. Couche presentation : le dossier `ui/`
10. Fichiers `.ui`
11. Base de donnees SQLite : schema complet
12. Roles et permissions
13. Construction des executables
14. Guide d'utilisation par role
15. Depannage et questions frequentes
16. Historique des versions

DOCUMENTATION COMPLETE DU LOGICIEL DE GESTION SCOLAIRE

Version documentee : 1.2.0 — Python 3 + PyQt5 + SQLite **Contexte :**
Republique du Congo (Brazzaville) — Devise : FCFA — Indicatif :
+242

Ce document decrit chaque fichier du projet, son role, et comment tout s'emboite pour former l'application. Les numeros de lignes correspondent au code source actuel (branche exe).

SOMMAIRE

1. [Vue d'ensemble](#)
 2. [Arborescence complete du projet](#)
 3. [Architecture et demarrage](#)
 4. [Point d'entree : main.py](#)
 5. [Configuration : core/config.py](#)
 6. [Couche donnees : database/db.py](#)
 7. [Depots : repositories/](#)
 8. [Services et API : services/ + api/](#)
 9. [Couche presentation : le dossier ui/](#)
 10. [Fichiers .ui](#)
 11. [Base de donnees SQLite : schema complet](#)
 12. [Roles et permissions](#)
 13. [Construction des executables](#)
 14. [Guide d'utilisation par role](#)
 15. [Depannage et questions frequentes](#)
 16. [Historique des versions](#)
-

1. Vue d'ensemble

Le logiciel **Gestion Scolaire** est une application de bureau autonome pour un etablissement scolaire congolais. Elle couvre :

- la **gestion des eleves** (inscription, reinscription, matricule automatique, dossier complet) ;
- la **gestion des classes** (capacite, salle, titulaire) ;
- la **saisie des notes** et la **generation de bulletins** ;
- l'**emploi du temps** (planning hebdomadaire) ;
- la **caisse** (recettes / depenses, solde, export CSV) ;
- la **gestion des comptes utilisateurs** (3 roles) ;
- la **gestion du personnel** (RH, salaires, paie) ;
- la **generation de documents** (bulletins, recus, certificats, paie, rapports RH, emploi du temps) au format **HTML** ouverts dans le navigateur ;

- des **dashboards** par role avec graphiques (barres, camemberts) dessines sans aucune dependance externe grace a QPainter.

Caracteristiques techniques

Element	Valeur
Langage	Python 3
Interface graphique	PyQt5 (5.15.11)
Base de donnees	SQLite (fichier <code>ecole.db</code>)
Backend optionnel	API REST FastAPI (<code>http://127.0.0.1:8000</code>) pour la synchronisation serveur (voir section 8.2)
Mode hors ligne	L'app fonctionne completement sans l'API : bascule sur la base locale
Mode Offline-First	En cas de serveur deployee, les ecritures passent par une file d'attente (<code>file_attente_synchro</code>) avant synchronisation
Empaquetage	PyInstaller one-file sur Linux et Windows
Dependances	PyQt5==5.15.11 et httpx==0.28.1 (voir <code>requirements.txt</code>)

Ou sont stockees les donnees ?

- En developpement : dossier `data/` a la racine du projet.
- En executable : dossier `data/` **a cote de l'executable** (voir `config.data_dir()`).

2. Arborescence complete du projet

```

Logiciel-de-gestion-scolaire/
|
├─ main.py                      # Point d'entree : HighDPI +
demarrage (65 lignes)
├─ requirements.txt             # PyQt5==5.15.11 + httpx==0.28.1
├─ .gitignore                   # venv/, .venv/, __pycache__/,
*.db, build/, dist/, dist_win/, exe-win/, data/,
GestionScolaire.spec
|
├─ build.spec                   # Spec PyInstaller LINUX (one-
file, sans console)

```

— build_win.spec	# Spec PyInstaller WINDOWS (one-
file + manifeste DPI)	
— win_dpi_manifest.xml	# Manifeste <dpiAware>true</
dpiAware> pour Windows	
— .github/workflows/	
— build_windows.yml	# Workflow GitHub Actions : build
de l'exe Windows	
— core/	# Configuration generale + etat
du reseau	
— __init__.py	
— config.py	# Constantes + feuille de style
(121 lignes)	
— network.py	# Etat reseau (online/offline) +
synchronisation active	
— database/	
— __init__.py	# Importe l'instance partagee de
la base	
— db.py	# Connexion SQLite, schema,
donnees initiales,	
	# file d'attente de
synchronisation (266 lignes)	
— repositories/	# Couche d'abstraction des
donnees (aiguillage reseau)	
— __init__.py	# Facade `repos` regroupant tous
les depots (39 lignes)	
— base.py	# RepositoryBase : file d'attente
+ _route_write (28 lignes)	
— eleve_repository.py	# EleveRepository (83 lignes)
— classe_repository.py	# ClasseRepository (39 lignes)
— pedagogie_repository.py	# PedagogieRepository : matieres,
enseignants (17 lignes)	
— finance_repository.py	# FinanceRepository : caisse (66
lignes)	
— note_repository.py	# NoteRepository (29 lignes)
— personnel_repository.py	# PersonnelRepository (43 lignes)
— compte_repository.py	# CompteRepository (43 lignes)
— planning_repository.py	# PlanningRepository (18 lignes)
— parametre_repository.py	# ParametreRepository + stats
dashboard (39 lignes)	
— services/	# Logique metier pure (pas de
PyQt)	
— __init__.py	# Importe l'instance partagee
AuthService	
— auth.py	# Connexion, mots de passe,
permissions (69 lignes)	
— reports.py	# Documents HTML (bulletins,
certificats, paie) (193 lignes)	

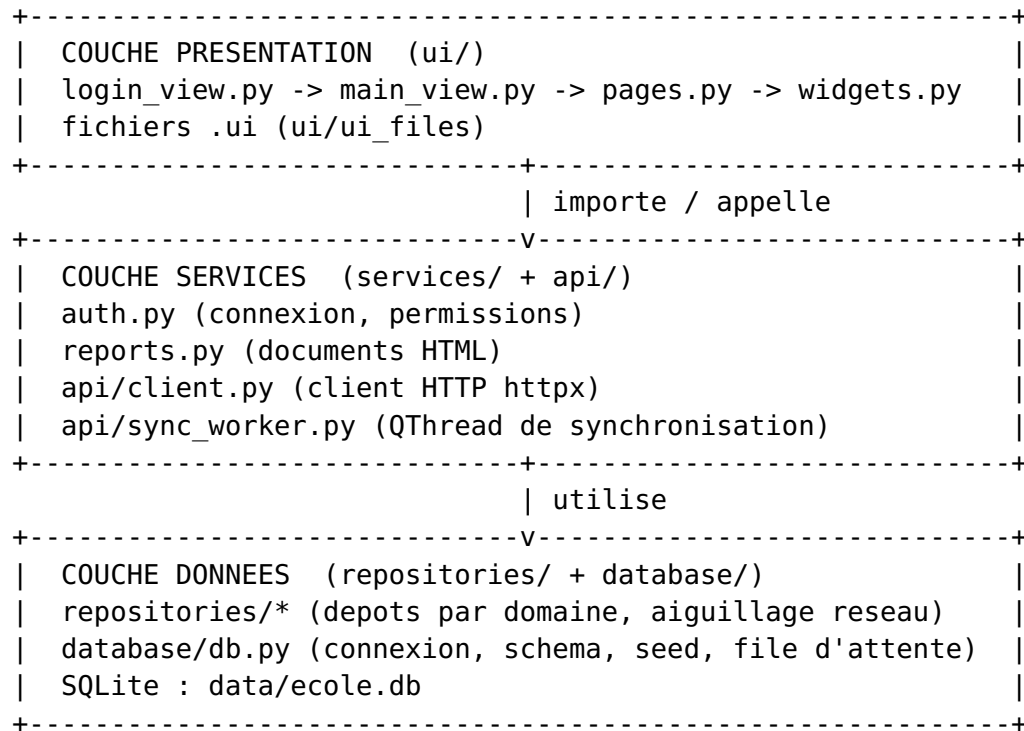
```

|
├─ api/                                # Client HTTP + filet de
synchronisation
|   └─ __init__.py                    # Re-exporte le client partage et
api_disponible
|   └─ client.py                      # Client httpx de l'API FastAPI +
stubs (442 lignes)
|   └─ sync_worker.py                # Squelette QThread : healthcheck
+ vidage de la file (39 lignes)
|
├─ ui/                                # Couche presentation PyQt5
|   └─ __init__.py                    # Importe load_ui
|   └─ loader.py                     # Chargement des .ui +
responsivite (26 lignes)
|   └─ login_view.py                 # Fenetre de connexion (125
lignes)
|   └─ main_view.py                  # Fenetre principale, sidebar,
statut / (191 lignes)
|   └─ pages.py                      # Toutes les pages et dialogues
metier (1455 lignes)
|   └─ widgets.py                    # Graphiques QPainter + formats
monetaires (176 lignes)
|   └─ ui_files/
|       └─ main.ui                   # Interfaces .ui (Qt Designer)
QStackedWidget)                     # Fenetre principale (sidebar +
|       └─ dashboards/               # dashboard_admin.ui,
dashboard_directeur.ui,
|       └─ eleves/                   # dashboard_gestionnaire.ui
|       └─ inscription.ui (dossier) # eleves.ui (liste) +
|           └─ classes/               # classes.ui + classe_dialog.ui
|           └─ notes/                 # notes.ui (saisie + bulletins)
|           └─ planning/              # planning.ui (emploi du temps)
|           └─ caisse/                # caisse.ui (recettes / depenses)
|           └─ comptes/               # comptes.ui + compte_dialog.ui
|           └─ parametres/            # parametres.ui (en-tete des
documents)
|
├─ data/                              # CREE A L'EXECUTION (base +
documents)
|   └─ ecole.db                      # La base SQLite
|   └─ documents/                    # HTML generes (bulletins, recus,
etc.)
|
├─ dist/GestionScolaire               # Executable Linux (one-file)
├─ dist_win/                          # Executable Windows (one-file,
via GitHub Actions)
├─ build/                             # Travail intermediaire
PyInstaller (nettoyable)
└─ .venv/                             # Environnement virtuel de
developpement

```

3. Architecture et démarrage

L'application suit une architecture en **3 couches**, sans framework :



Demarrage pas a pas

1. python main.py est execute.
2. main() (main.py:43):
 - configure le **HighDPI** (_setup_high_dpi, main.py:15);
 - choisit une police vectorielle (_pick_base_font, main.py:23);
 - installe _excepthook (main.py:34) : boite de dialogue au lieu d'un crash silencieux en executable ;
 - cree l'application Qt et applique APP_STYLESHEET (core/config.py: 29);
 - appelle db.init_db() -> creation des tables + donnees initiales ;
 - ouvre la **fenetre de connexion** LoginDialog ;
 - si la connexion reussit -> ouvre MainWindow en plein ecran ;
 - sinon -> quitte.

Navigation dans la fenetre principale

- MainWindow charge main.ui : une **sidebar** de boutons et un QStackedWidget pour le contenu.
- _wire_nav() (main_view.py:128) affiche uniquement les boutons autorises pour le role connecte.

- `_build_pages()` (`main_view.py:138`) construit chaque page autorisee dans un `QWidget` entoure d'une `QScrollArea`, et l'ajoute au `QStackedWidget`. En cas d'erreur, un libelle rouge s'affiche au lieu de faire planter l'app.
- `navigate(page)` (`main_view.py:173`) bascule la page, appelle son `refresh()` et coche le bouton actif.

Rafrachissement

Chaque page expose `page.refresh = refresh`. A chaque navigation, `MainWindow.navigate` appelle `refresh()` pour recharger les donnees depuis la base. Les pages restent en memoire et se rafraichissent a la demande.

4. Point d'entree : `main.py`

`main.py` (65 lignes)

Ligne	Element	Role
15-22	<code>_setup_high_dpi()</code>	Rendu net sur les ecrans Windows 100/125/150/200 % (variables d'environnement Qt posees avant <code>QApplication</code>).
23-33	<code>_pick_base_font()</code>	Police vectorielle presente sur le systeme (<code>APP_FONT_FAMILY</code> , puis fallbacks), sinon police generique.
34-42	<code>_excepthook(...)</code>	En executable, une exception non geree affiche une boite « Erreur inattendue » avec le detail.
43-62	<code>main()</code>	Orchestration complete du demarrage (voir section 3).
64-65	garde	<code>if __name__ == "__main__": main()</code>

5. Configuration : `core/config.py`

`core/config.py` (121 lignes)

Module importe partout : constantes, chemins, feuille de style.

Constantes principales

Constante	Valeur	Ligne	Role
APP_NAME	"Gestion Scolaire"	6	Titre de l'application
APP_VERSION	"1.2.0"	7	Version affichee sur l'ecran de connexion
APP_FONT_FAMILY / APP_FONT_FALLBACK / APP_FONT_SIZE	"Segoe UI" / tuples / 10	14-16	Police et taille du texte
API_BASE_URL	"http:// 127.0.0.1:8000"	19	Adresse du backend FastAPI (optionnel)
API_TIMEOUT	3.0	20	Delai d'attente avant bascule sur la base locale
PAYS_DEFAULT / VILLE_DEFAULT	"Republique du Congo" / "Brazzaville"	23-24	Valeurs par default des documents
INDICATIF_TEL / DEVISE	" +242" / "FCFA"	25-26	Contexte congolais
APP_STYLESHEET	CSS Qt	29-46	Feuille de style globale de l'app
ROLES	("admin", "directeur", "gestionnaire")	49	Roles possibles
ROLE_LABELS	{"admin": "Administrateur", ...}	51	Libelles affichees
PERIODES	("1er Trimestre", ...)	58	Periodes de notes
JOURS	("Lundi", ..., "Samedi")	60	Jours du planning
CRENEAUX	8 creneaux	62	Horaires des cours
DEFAULT_ACCOUNTS	3 comptes	97	

Constante	Valeur	Ligne	Role
			Comptes crees au premier lancement (voir section 11)
DEFAULT_MATIERES	8 matieres	110	Francais, Maths, Anglais, HG, SVT, PC, Educ. civique, Informatique

Fonctions de chemins

Fonction	Ligne	Role
PROJECT_ROOT	74	Racine du projet (Path(__file__).resolve().parent.parent) : core/ est un niveau sous la racine.
resource_path(relative)	77	Chemin vers les donnees empaquetees dans l'executable (_MEIPASS), sinon PROJECT_ROOT.
data_dir()	83	Dossier ecrivable ou vit la base : a cote de l'executable, sinon PROJECT_ROOT. Cree data/ si besoin.

Constantes de chemins

Constante	Ligne
UI_DIR (dossier des .ui)	92
DB_PATH (fichier data/ecole.db)	93
DOCS_DIR (dossier data/documents/)	94

En fin de module, `os.makedirs(data_dir(), exist_ok=True)` garantit que le dossier `data/` existe des le chargement de config.

6. Couche donnees : database/db.py

database/db.py (266 lignes)

Fonctions et schema

Element	Ligne	Role
SCHEMA	7	Script SQL de creation des tables, dont file_attente_synchro (voir section 11).
hash_password(password)	130	Retourne le hash sha256 hexadecimal. Les mots de passe ne sont jamais stockes en clair.

Classe Database (singleton)

Une seule instance partagee par toute l'app.

Methode	Ligne	Role
__new__	134	Retourne l'unique instance
__init__	139	Prepare le dossier des documents, pose _initialized = False
connect()	148	Ouvre une connexion SQLite (row_factory = Row -> dictionnaires), active PRAGMA foreign_keys = ON. Une connexion par operation.
init_db()	156	Execute SCHEMA puis _seed, une seule fois.
_seed(conn)	169	Comptes par defaut (si aucun utilisateur), matieres, parametres, 3 membres de personnel de base.
query(sql, params)	213	SELECT -> liste de dictionnaires.
query_one(sql, params)	222	Premier enregistrement ou None.
execute(sql, params)	227	INSERT/UPDATE/DELETE, commit, retourne lastrowid.

Methode	Ligne	Role
<code>executemany(sql, seq)</code>	237	Execution en lot, commit.
<code>enqueue(method, endpoint, payload)</code>	246	Ecrit une operation dans <code>file_attente_synchro</code> (statut PENDING).
<code>dequeue_pending(limit=50)</code>	253	Retire les operations en attente (a envoyer au serveur).
<code>mark_queue_done(queue_id)</code>	259	Supprime une operation envoyee avec succes.
<code>mark_queue_failed(queue_id)</code>	263	Marque FAILED une operation qui a echoue (relancee plus tard).

Le `_seed` cree uniquement les donnees de base (comptes, matieres, parametres, personnel). Les eleves, transactions et notes sont saisis par l'utilisateur dans l'application.

7. Depots : repositories/

repositories/ (10 fichiers, 444 lignes)

La couche **repositories** abstrait l'acces aux donnees. Chaque domaine a son propre depot, et tous sont regroupes derriere la facade repos (`from repositories import repos`). Chaque ecriture passe par `_route_write`, qui decide selon l'etat du reseau (voir section 8.4) :

- **reseau disponible** -> appel a l'API serveur (TODO, stub) ;
- **reseau coupe** -> base locale, et operation mise en file d'attente.

repositories/base.py — RepositoryBase

Element	Ligne	Role
<code>_gen_reference(prefix)</code>	10	Reference unique de transaction REC-XXXXXXX / DEP-XXXXXXX.
<code>_enqueue(method, endpoint, payload)</code>	16	Pousse une operation dans <code>db.enqueue</code> quand la synchronisation est active.
	21	

Element	Ligne	Role
<code>_route_write(method, endpoint, payload, fn, ...)</code>		Aiguillage reseau : en ligne -> stub API ; hors ligne -> appelle fn localement puis <code>_enqueue</code> .

repositories/eleve_repository.py — EleveRepository

Methode	Ligne	Role
<code>eleves(classe_id, statut, recherche)</code>	8	Liste filtree (classe, statut, texte) avec <code>classe_nom</code> via LEFT JOIN.
<code>eleve_by_id(id)</code>	26	Un eleve.
<code>eleve_by_matricule(m)</code>	29	Un eleve par matricule (reinscription).
<code>add_eleve(data)</code>	32	Insere un eleve ; genere le matricule si absent.
<code>update_eleve(id, data)</code>	51	Met a jour le dossier.
<code>delete_eleve(id)</code>	64	Supprime notes puis eleve.
<code>next_matricule()</code>	69	Prochain matricule <code>ELEV{annee}{NNNN}</code> a partir du dernier existant.

repositories/classe_repository.py — ClasseRepository

Methode	Ligne	Role
<code>classes()</code>	8	Toutes les classes + effectif reel (COUNT sur eleves), trieées par nom.
<code>classe_by_id(id)</code>	13	Une classe.
<code>add_classe(...)</code>	16	Insere une classe.
<code>update_classe(...)</code>	25	Modifie une classe.
<code>delete_classe(id)</code>	34	Supprime d'abord ce qui depend de la classe (eleves, planning), puis la classe.

repositories/pedagogie_repository.py — PedagogieRepository

Methode	Ligne	Role
matieres()	8	Toutes les matieres trieés.
add_matiere(nom)	11	Ajoute une matiere (IGNORE si doublon).
enseignants()	16	Personnel dont la fonction contient « Enseignant ».

repositories/note_repository.py — NoteRepository

Methode	Ligne	Role
notes_for(classe_id, matiere_id, periode)	8	Notes d'une classe x matiere x periode (avec matricule et nom).
save_note(...)	17	Insere ou met a jour (UPSERT) sur (eleve_id, matiere_id, periode).

repositories/finance_repository.py — FinanceRepository

Methode	Ligne	Role
transactions(type, recherche, dates)	10	Liste filtree, tri date DESC.
add_transaction(...)	31	Insere avec reference auto REC-.../DEP-....
delete_transaction(id)	45	Supprime.
caisse_totals()	49	Somme des entrees, des sorties, solde.
export_transactions_csv(path)	59	CSV separe par ; avec BOM (utf-8-sig) pour Excel.

repositories/personnel_repository.py — PersonnelRepository

Methode	Ligne	Role
personnel(recherche)	8	Liste du personnel.

Method	Ligne	Role
add_personnel(...)	17	Ajoute.
update_personnel(...)	27	Modifie.
delete_personnel(id)	37	Supprime.
masse_salariale()	41	Somme des salaires.

repositories/compte_repository.py — CompteRepository

Method	Ligne	Role
utilisateurs(role, recherche)	8	Liste (sans le hash de mot de passe).
add_compte(...)	22	Cree un compte ; username derive de l'email.
update_compte(...)	29	Modifie identite/email/telephone/role/actif.
toggle_compte(id, actif)	35	Active/desactive.
reset_password(id, hash)	38	Remplace le mot de passe.
delete_compte(id)	41	Supprime connexions puis compte.

repositories/planning_repository.py — PlanningRepository

Method	Ligne	Role
planning_for(classe_id)	8	Dictionnaire {(jour, creneau): entree}.
save_planning(classe_id, entries)	14	Remplace tout le planning d'une classe.

repositories/parametre_repository.py — ParametreRepository

Method	Ligne	Role
parametres()	8	Dictionnaire cle -> valeur.
set_parametre(cle, valeur)	13	Upsert d'un parametre.
delete_parametres()	18	Reinitialise signataire, titre, ville.
stats_dashboard()	23	

Method	Ligne	Role
		Total eleves, inscriptions du jour, encaissements du jour, dossiers incomplets.

repositories/__init__.py — la facade repos

Element	Ligne	Role
_Repos	14	Classe aplat : chaque attribut eleves, classes, notes, finance, personnel, comptes, planning, parametres designe le depot du domaine.
__getattr__(name)	32	Retourne le depot demande (cree a la demande).
repos = _Repos()	39	Instance partagee importee partout (from repositories import repos).

Le nom d'attribut personnel_repo evite la collision entre le depot personnel et la methode personnel() qui liste les membres.

8. Services et API : services/ + api/

8.1 services/auth.py (69 lignes) — connexion et permissions

Classe AuthService

Method	Ligne	Role
login(username, password)	10	Recherche par username ou email ; verifie activite puis mot de passe (hash) ; met a jour last_login et journalise la connexion. Retourne (user, None) ou (None, message).
change_password(user_id, old, new)	27	Verifie l'ancien mot de passe puis met a jour.
random_password()	36	

Methode	Ligne	Role
		Mot de passe temporaire aleatoire (12 caracteres hexa).
<code>derniere_connexions(limit=20)</code>	40	Journal des connexions (nom, role, date).

Classe `RoleAuthorizer` (permissions par role)

- NAV (ligne 48) : pages autorisees par role :
 - admin : dashboard, comptes ;
 - directeur : dashboard, eleves, classes, notes, planning, caisse, personnel, parametres ;
 - gestionnaire : dashboard, eleves, classes, notes, planning, caisse.
- `allowed(page)` (60) : la page est-elle autorisee ?
- `can_edit(page)` (64) : qui peut saisir ? comptes -> seul admin ; sinon -> le gestionnaire oui, les autres non.

8.2 `api/client.py` (442 lignes) — client de l'API FastAPI

L'app peut communiquer avec un backend REST FastAPI (<http://127.0.0.1:8000>). Si l'API est hors ligne, elle **bascule automatiquement sur la base locale** : l'utilisateur ne voit aucune erreur. Les methodes sont decoupees en stubs de routes (la liste complete est donnee dans la section 8.4).

Element	Ligne	Role
<code>ApiError</code>	8	Exception interne (jamais exposee a l'interface).
<code>_request(method, path, **kwargs)</code>	13	Envoie une requete HTTP (librairie httpx) et normalise (donnees, erreur). Toutes les erreurs reseau sont interceptees.
<code>_CacheDispo</code>	35	Memorise pendant 5 s si l'API repond (evite de tester a chaque appel).
<code>api_disponible(force=False)</code>	51	L'API est-elle joignable ? (healthcheck sur / annee_scolaire_active).
<code>ApiClient</code>	58	

Element	Ligne	Role
		Un client par domaine : eleves, classes, matieres, paiements, enseignants, notes, presences, annees scolaires. Chaque methode renvoie (donnees, erreur).
client	442	Instance unique utilisee par les dashboards et les depots.

Exemples de methodes : `total_eleves()`, `eleve_recherche(...)`,
`total_montant_paiement()`, `paiements()`, `enseignants()`,
`ajouter_eleve(...)`, `cycle_par_id(...)`, `tarifs_scolarite()`,
`bilan_eleve(...)`, `ajouter_note(...)`.

Attention : `api/__init__.py` re-exporte `client = ApiClient()`. En
Python, cet attribut masque le sous-module `api.client` ; pour
tester en patchant `API_BASE_URL`, passer par
`importlib.import_module("api.client")`.

8.3 services/reports.py (193 lignes) — documents HTML

Tous les documents sont generes en **HTML** dans `DOCS_DIR` puis **ouverts dans le navigateur** (`QDesktopServices.openUrl()`). Pas de dependance PDF.

Element	Ligne	Role
STYLE	9	CSS des documents (tableaux bordes, en-tete vert #047857).
<code>_open_in_browser(path)</code>	19	Ouvre un fichier local dans le navigateur.
<code>_write(title, body, filename)</code>	25	Ecrit le HTML complet et l'ouvre.
<code>_entete_doc()</code>	34	En-tete generique (nom du logiciel, pays/ville, date).
<code>export_eleves_csv(eleves)</code>	45	CSV (;, BOM) dans <code>data/</code> .
<code>bulletins(classe_id, periode)</code>	59	Bulletins d'une classe. Moyenne par matiere : $(d1 + d2 + 2 \times comp) / 4$, moyenne generale ponderee.

Element	Ligne	Role
recu_paiement(...)	98	Recu de paiement.
certificat_scolarite(eleve, params)	117	Certificat avec le signataire (parametres).
paie()	137	Bulletins de paie du mois + masse salariale.
rapport_rh()	152	Rapport RH mensuel.
planning(classe)	165	Emploi du temps d'une classe.
_appreciation(moyenne)	181	Mention : >=16 Excellent, >=14 Tres bien, >=12 Bien, >=10 Assez bien, >=8 Passable, sinon Insuffisant.

8.4 Mode Offline-First : routes, reseau et synchronisation

Le projet prepare un **Stack Serveur FastAPI + MySQL** (port 8000). Les routes sont definies cote client sous forme de **stubs** dans `ApiClient` : elles renvoient (`None`, "API hors ligne (`ConnectError`)") tant que le serveur n'est pas deploye, puis (`404`, ...) quand il repond. Liste des stubs ajoutes :

Domaine	Methodes
Cycles	<code>cycle_par_id(cycle_id)</code>
Matieres	<code>matiere_par_id(matiere_id)</code>
Scolarite	<code>tarifs_scolarite()</code> , <code>tarifs_scolarite_classe(classe_id)</code> , <code>ajouter_tarif(...)</code> , <code>modifier_tarif(...)</code> , <code>supprimer_tarif(...)</code>
Inscriptions	<code>solde_inscription(...)</code> , <code>suivi_mensuel_inscription(...)</code> , <code>bilan_eleve(...)</code>
Notes	<code>ajouter_note(...)</code> , <code>modifier_note(...)</code> , <code>supprimer_note(...)</code>

L'aiguillage reseau est assure par `core/network.py` :

Element	Ligne	Role
<code>SYNC_ACTIVE</code>	11	False par default (le serveur n'est pas encore deploye). Quand il le sera, le passer a True.

Element	Ligne	Role
_state	12	Etat courant : online ou offline.
sync_active()	15	La synchronisation serveur est-elle activee ?
is_online()	20	Le reseau est-il disponible ?
set_online() / set_offline()	25 / 31	Bascule l'etat (utilise par le SyncWorker et les reponses de l'API).

api/sync_worker.py (39 lignes) : squelette de SyncWorker(QThread) avec les signaux status_changed, sync_done, sync_error. Son run() teste l'API (api_disponible(force=True)), met a jour l'etat reseau, puis vide la file d'attente (_drain_queue -> db.dequeue_pending). Le transfert reel des operations est a implementer (TODO dans le code).

Chaine d'une ecriture en mode Offline-First :

1. L'interface appelle un depot (ex. repos.eleves.add_eleve(...)).
2. RepositoryBase._route_write (base.py:21) regarde net.is_online() :
 - **en ligne** -> appelle le stub API correspondant ;
 - **hors ligne** -> execute la requete en base locale, puis _enqueue (base.py:16) pousse (method, endpoint, payload) dans file_attente_synchro avec le statut PENDING.
3. Au retour du reseau, SyncWorker vide la file : chaque operation est envoyee au serveur, puis mark_queue_done la supprime (ou mark_queue_failed la garde en FAILED pour reessai).

9. Couche presentation : le dossier ui/

9.1 ui/loader.py (26 lignes)

Fonction	Ligne	Role
_make_responsive(widget)	8	Applique QSizePolicy.Expanding uniquement a la page racine (volontaire : l'expansion forcee sur les enfants etire les boutons en blocs geants).
load_ui(relative_path, widget=None)	18	Charge un .ui via PyQt5.uic.loadUi, rend la page responsive.

Fonction	Ligne	Role
apply_ui(relative_path, widget)		Applique un .ui a un widget existant (MainWindow, dialogues).

9.2 ui/main_view.py (191 lignes)

Element	Ligne	Role
NAV_PAGES	9	Correspondance bouton sidebar -> page (dashboard, eleves, caisse, classes, notes, planning, personnel, parametres, comptes).
DASHBOARD_BUILDERS	21	role -> fabrique de dashboard (admin/directeur/gestionnaire).
BUILDERS	27	page -> fabrique (eleves, caisse, classes, notes, planning, personnel, parametres, comptes).

Classe MainWindow(QMainWindow) (ligne 39) :

Methode	Ligne	Role
__init__(user)	40	Stocke l'utilisateur, cree PageContext, applique main.ui, remplit les labels, cree le label de statut du reseau, cable la navigation, construit les pages, navigue vers dashboard.
_refresh_api_status()	106	Interroge api.api_disponible() et met a jour la barre d'etat : « En Ligne » quand le serveur repond, « Mode Local » sinon.
logout()	123	Confirme puis ferme la fenetre.
_wire_nav()	128	Affiche les boutons autorises, masque les autres.
_build_pages()	138	Construit les pages autorisees dans des QScrollArea ; toute erreur affiche un libelle rouge.
navigate(page_name)	173	Bascule la page, appelle refresh(), coche le bouton.

9.3 ui/login_view.py (125 lignes)

Element	Ligne	Role
PRIMARY / PRIMARY_DARK	11-12	Couleurs du bouton de connexion (#047857 / #065f46).
_Gradient(QFrame)	15	Arriere-plan en degrade peint via QPainter.paintEvent.
LoginDialog(QDialog)	25	Dialogue de connexion 700x540.
_build()	34	Construit l'interface en code : carte blanche arrondie, champs nom/mot de passe, bouton degrade, rappel des comptes de test.
_do_login()	112	Appelle auth.login, affiche l'erreur si besoin, sinon stocke self.user et accept().

9.4 ui/pages.py (1455 lignes) — les pages metier

C'est le cœur de l'interface. Chaque fabrique suit le meme contrat : fabrique(page, ctx) remplit le widget page, connecte les signaux et expose page.refresh.

Contexte commun

Element	Ligne	Role
PageContext	22	Wrapper : user, role, authorizer, navigate, can_edit(page).

Utilitaires

Fonction	Ligne	Role
_btn(text, callback, style)	33	Bouton compact (hauteur max 28 px, curseur main).
_simple_btn_style(...)	42	Style de bouton discret (fond clair, bordure).
_money_edit(...)	46	Champ montant en FCFA.
_today_fr()	55	Date du jour en francais.
_fill_combos(combo, items)	62	Remplit une combo.

Dashboards

Fabrique	Ligne	Role
dashboard_admin	69	KPIs des comptes (actifs, inactifs), dernieres connexions, actions rapides (nouveau compte, reinitialiser un mot de passe).
dashboard_directeur	99	Masse salariale, enseignants (API), taux de scolarite (API), graphiques (tresorerie 6 mois, effectifs par classe, personnel), validations (classes sans titulaire -> double-clic), actions (paie, rapport RH, auditer la caisse, recrutement).
dashboard_gestionnaire	204	KPIs (eleves, inscriptions du jour, encaissements du jour, dossiers incomplets), activite recente, dossiers incomplets, graphique des statuts, actions rapides (inscrire, recette, depense, certificat).
_directeur_charts()	175	Donnees des graphiques du directeur (6 mois, 6 premieres classes).
_replace_layout(layout, widget)	195	Remplace le contenu d'un layout par un graphique.

Les KPIs des dashboards utilisent l'API **quand elle est disponible** (ex. total_eleves, total_montant_paiement, total_enseignant), sinon les valeurs de la base locale.

Page Eleves

Element	Ligne	Role
eleves(page, ctx)	269	Tableau des eleves. Par default, une classe est preselectionnee (la premiere) ; l'option « Toutes les classes » reste disponible. Filtres classe/statut/recherche,

Element	Ligne	Role
		boutons Modifier/ Supprimer par ligne, double-clic pour modifier, export CSV, 4 KPIs recalculés sur la classe choisie.
open_inscription_dialog(parent, ctx, eleve=None)	348	Dialogue d'inscription (1000x780) : aperçu du matricule, reinscription par recherche de matricule, classe (création à la volée), dossier complet (naissance, parents, adresse, documents), montant verse + mode de règlement (encaisse automatiquement, reçu imprimable).
_parse_money(text)	531	Parse un montant saisi (espaces, virgules, « FCFA »).

Page Classes

Element	Ligne	Role
classes(page, ctx)	538	Tableau des classes (nom, effectif, capacité, titulaire, salle) ; alerte rouge quand la classe est pleine ; recherche ; 4 KPIs.
open_classe_dialog(parent, ctx, classe=None, on_created=None)	606	Créer/modifier une classe (nom, niveau, capacité, salle, titulaire).

Page Notes

Element	Ligne	Role
notes(page, ctx)	654	Saisie par classe x matière x période : tableau éditable,

Element	Ligne	Role
		moyenne (d1+d2+2xcomp)/4 et appreciation calculees en direct, bornage 0-20, ajout de matiere, generation des bulletins. Les boutons de saisie sont masques si le role ne peut pas saisir.
_appreciation(moyenne)	800	Mention locale (identique a reports._appreciation).

Page Planning

Element	Ligne	Role
PlanningCellDialog	813	Dialogue pour affecter matiere + salle a une case.
planning(page, ctx)	845	Grille jours (6 colonnes) x creneaux (8 lignes). Mode edition : double-clic pour affecter, « Enregistrer le Planning » reecrit tout. Bouton d'impression -> HTML.

Page Caisse

Element	Ligne	Role
caisse(page, ctx)	959	Tableau des transactions filtre par type, recherche et dates (mois courant par default). Colonnes Entree/Sortie separees, total entrees, total sorties, solde. Export CSV. Boutons Recette/Depense masques si le role ne saisit pas.
open_transaction_dialog(parent, ctx, type_trans)	1037	Saisie motif, beneficiaire, montant, categorie, mode de reglement.

Page Comptes (admin)

Element	Ligne	Role
comptes(page, ctx)	1083	Tableau des comptes (nom, email, role, actif), filtres, actions par ligne (Activer/ Desactiver, Mdp, Supprimer), 4 KPIs.
open_compte_dialog(parent, ctx, compte=None)	1148	Creer/modifier un compte ; mot de passe temporaire genere et affiche.
open_change_password_dialog(parent, user)	1194	Changer son propre mot de passe.
open_reset_password_dialog(parent, ctx)	1223	Reinitialiser le mot de passe d'un compte non-admin.

Page Personnel

Element	Ligne	Role
personnel(page, ctx)	1249	Tableau du personnel construit en code (nom, fonction, telephone, email, salaire), recherche, « + Nouvel Employe ».
open_personnel_dialog(parent, ctx, employe=None)	1320	Creer/modifier un employe (fonction, salaire, statut Contrat/ CDI/Vacataire/Stage).

Certificat

Element	Ligne	Role
open_certificat_dialog(parent)	1366	Dialogue filtre par classe : choisir la classe puis l'eleve, et genere le certificat de scolarite.

Parametres

Element	Ligne	Role
parametres(page, ctx)	1403	Signataire (nom/titre), ville, pays + images (bandeau haut/bas, signature) copiees dans DOCS_DIR. Bouton supprimer la configuration.

9.5 ui/widgets.py (176 lignes) — graphiques et formats

Element	Ligne	Role
CHART_COLORS	8	Palette de couleurs des graphiques.
_color(i)	13	Couleur par index (cycle sur la palette).
fmt_money(montant)	17	1 234 567 FCFA (separateur d'espace).
fmt_money_short(montant)	24	Sans le suffixe « FCFA ».
_BaseChart(QWidget)	28	Base commune des graphiques (marge, titre, anti-aliasing).
SimpleBarChart	70	Histogramme (valeurs au-dessus des barres, echelle Y). set_data(labels, values).
SimplePieChart	117	Camembert (pourcentages, legende). set_data(labels, values).

10. Fichiers .ui

Les interfaces sont definies dans ui/ui_files/ et chargees par apply_ui. Les fichiers reellement utilises :

Fichier	Charge par	Role
main.ui	main_view.py: 83	Fenetre principale (sidebar + stackedWidget + labels + bouton deconnexion).

Fichier	Charge par	Role
dashboards/ dashboard_admin.ui	pages.py:70	Accueil admin.
dashboards/ dashboard_directeur.ui	pages.py:100	Accueil directeur.
dashboards/ dashboard_gestionnaire.ui	pages.py:205	Accueil gestionnaire.
eleves/eleves.ui	pages.py:270	Liste des eleves.
eleves/inscription.ui	pages.py:352	Dossier d'inscription.
classes/classes.ui	pages.py:539	Liste des classes.
classes/classe_dialog.ui	pages.py:610	Dialogue classe.
notes/notes.ui	pages.py:655	Saisie des notes.
planning/planning.ui	pages.py:846	Emploi du temps.
caisse/caisse.ui	pages.py:960	Caisse.
comptes/comptes.ui	pages.py:1084	Liste des comptes.
comptes/compte_dialog.ui	pages.py:1152	Dialogue compte.
parametres/parametres.ui	pages.py:1404	Parametres des documents.

11. Base de donnees SQLite : schema complet

Fichier : data/ecole.db. Cree par SCHEMA (db.py:7).

Tables

utilisateurs — comptes de connexion

Colonne	Type	Contrainte
id	INTEGER	PK AUTOINCREMENT
nom_complet	TEXT	NOT NULL
username	TEXT	NOT NULL UNIQUE
email	TEXT	—
telephone	TEXT	—
password	TEXT	NOT NULL (hash sha256)
role	TEXT	NOT NULL
actif	INTEGER	NOT NULL DEFAULT 1

Colonne	Type	Contrainte
created_at	TEXT	DEFAULT datetime('now','localtime')
last_login	TEXT	—

classes : id, nom (NOT NULL), niveau, capacite (INTEGER DEFAULT 50), salle, titulaire

matieres : id, nom (TEXT NOT NULL UNIQUE), coefficient (REAL DEFAULT 1)

elevés : id, matricule (UNIQUE), nom (NOT NULL), prenom (NOT NULL), sexe, date_naissance, lieu_naissance, classe_id (FK -> classes), ecole_provenance, pere_nom, pere_tel, mere_nom, mere_tel, tuteur_nom, tuteur_tel, adresse, check_acte / check_photos / check_bulletin (INTEGER DEFAULT 0), statut (TEXT DEFAULT 'Inscrit'), date_inscription (TEXT DEFAULT date('now','localtime'))

notes : id, eleve_id (FK), matiere_id (FK), periode (TEXT), devoir1 / devoir2 / composition (REAL), **UNIQUE(eleve_id, matiere_id, periode)**

transactions : id, date (DEFAULT aujourd'hui), reference (NOT NULL), beneficiaire, motif, categorie, montant (REAL NOT NULL), type (CHECK IN ('entree','sortie')), mode_reglement

planning : id, classe_id (FK), jour (TEXT), creneau (TEXT), matiere, salle, **UNIQUE(classe_id, jour, creneau)**

connexions : id, utilisateur_id (FK), date_connexion (DEFAULT maintenant) -> journal d'audit des connexions.

personnel : id, nom_complet (NOT NULL), fonction, telephone, email, salaire (REAL DEFAULT 0), statut (DEFAULT 'Contrat')

parametres : cle (TEXT PK), valeur (TEXT) -> paires cle/valeur (signataire_nom, signataire_titre, ville, pays, frais_scolarite, bandeau_haut, bandeau_bas, signature).

file_attente_synchro — file d'attente du mode Offline-First

Colonne	Type	Contrainte
id	INTEGER	PK AUTOINCREMENT
endpoint	TEXT	NOT NULL (route du serveur, ex. /eleve)
method	TEXT	NOT NULL (POST, PUT, DELETE)
payload	TEXT	NOT NULL (donnees JSON serialisees)

Colonne	Type	Contrainte
created_at	TEXT	DEFAULT datetime('now','localtime')
status	TEXT	DEFAULT PENDING (ou FAILED apres un echec)

Comptes par default (crees au premier lancement)

Nom	Identifiant	Mot de passe	Role
Administrateur Systeme	admin	admin123	admin
Directeur de l'Ecole	directeur	directeur123	directeur
Gestionnaire Scolaire	gestionnaire	gestionnaire123	gestionnaire

A **changer** des la mise en production reelle.

12. Roles et permissions

Action	admin	directeur	gestionnaire
Voir le dashboard	OUI (comptes)	OUI	OUI
Gerer les comptes	OUI	NON	NON
Gerer les eleves	NON	consulter	saisir
Gerer les classes	NON	consulter	saisir
Saisir les notes	NON	NON	OUI
Gerer le planning	NON	consulter	saisir
Caisse (recettes/ depenses)	NON	NON	OUI
Personnel / RH	NON	OUI (paie, rapports)	NON
Parametres des documents	NON	OUI	NON

- Navigation : RoleAuthorizer.NAV (auth.py:48) — les boutons de la sidebar sont masques selon le role (main_view.py:128).
- Saisie : RoleAuthorizer.can_edit (auth.py:64) — les boutons d'ajout et de modification sont masques quand le role ne peut pas saisir.

13. Construction des executables

Deux spec PyInstaller **one-file** sont versionnees : `build.spec` (Linux) et `build_win.spec` (Windows). Les deux embarquent `ui/ui_files`. Le dossier `ui/ui_files` doit toujours etre au meme emplacement relatif que dans la source : les chemins sont calcules via `PROJECT_ROOT` (`core/config.py:74`), qui remonte d'un niveau depuis `core/`.

Executable Linux

```
python3 -m venv venv
venv/bin/pip install -r requirements.txt
venv/bin/pyinstaller --noconfirm --clean build.spec
```

`build.spec` produit `dist/GestionScolaire` (one-file, sans console).
Hidden imports : `PyQt5.uic`, `PyQt5.uic.plugins`, `PyQt5.QtSql`.

Executable Windows (via GitHub Actions)

Le workflow `.github/workflows/build_windows.yml` construit l'exe Windows sur chaque push vers la branche `exe` :

1. checkout du code ;
2. `setup-python 3.11` ;
3. installation de `requirements.txt` + `pyinstaller` ;
4. `pyinstaller --noconfirm --clean build_win.spec` (one-file, sans console) ;
5. depose de `dist/GestionScolaire.exe` comme **artefact** de la run.

L'artefact est telechargeable depuis la page Actions du depot GitHub (Actions -> Build Windows Executable -> Artifact).

Spec Windows (`build_win.spec`)

`build_win.spec` construit un **one-file** nomme `GestionScolaire.exe`, sans console, en embarquant `ui/ui_files` et le manifeste DPI.

Manifeste DPI Windows (`win_dpi_manifest.xml`)

Le manifeste declare `<dpiAware>true</dpiAware>`. Il est **embarque dans l'exe** par `build_win.spec` (`EXE(manifest=...)`). Combine au `_setup_high_dpi` de `main.py`, il rend le texte net sur les ecrans Windows 100/125/150/200 %.

Le fichier n'est pas signe : Windows SmartScreen peut afficher un avertissement au premier lancement (« Informations supplémentaires -> Executer quand meme »).

Verification apres build

- Lancer l'executable : il doit rester vivant et creer data/ecole.db.
 - Windows : tester avec QT_SCALE_FACTOR=1.25 pour simuler un ecran 125 %.
-

14. Guide d'utilisation par role

Connexion

Identifiants de test : admin/admin123, directeur/directeur123, gestionnaire/gestionnaire123 (rappelles sur l'ecran de connexion).

Gestionnaire (saisie quotidienne)

1. **Inscription** : dashboard -> « Inscrire » -> remplir le dossier (nom, prenom obligatoires, classe obligatoire) -> montant verse eventuel -> reçu imprimable. Reinscription : saisir le matricule -> « Rechercher ».
2. **Eleves** : la liste s'ouvre sur la **premiere classe** ; changer la classe avec la liste deroulante ou choisir « Toutes les classes ». Les chiffres (KPIs) se mettent a jour sur la classe choisie.
3. **Classes** : creer/modifier (nom, niveau, capacite, salle, titulaire).
4. **Notes** : choisir classe, matiere, periode -> « Charger » -> saisir -> « Enregistrer les notes ». Moyenne et appreciation calculees en direct.
5. **Planning** : choisir la classe -> « Modifier » -> double-cliquer sur une case pour affecter matiere/salle -> « Enregistrer le Planning ».
6. **Caisse** : « + Recette » / « + Depense ». Filtres par type, texte et dates. Export CSV.
7. **Certificat** : dashboard -> « Certificat » -> choisir la **classe** puis l'**eleve** -> « Generer ».

Directeur (supervision)

- Dashboard : masse salariale, taux de scolarite, graphiques, validations (classes sans titulaire -> double-clic pour affecter), actions « Paie », « Rapport RH », « Auditer la caisse ».
- Consulte eleves/classes/notes/planning/caisse/personnel/parametres **sans pouvoir saisir** (les boutons d'edition sont masques).
- Parametres : renseigne le signataire (nom/titre), la ville, le pays et les images d'en-tete/signature des documents.

Admin (comptes uniquement)

- Dashboard : comptes actifs/inactifs, dernieres connexions.
- Page Comptes : creer un compte (mot de passe temporaire affiche), activer/desactiver, reinitialiser le mot de passe, supprimer.
- Ne voit ni les eleves, ni la caisse, ni les notes (sidebar reduite).

Documents generes (HTML ouverts dans le navigateur)

Bulletins (page Notes), reçu de paiement (inscription), certificat de scolarite, paie + rapport RH (dashboard directeur), emploi du temps (planning). Les fichiers sont stockes dans data/documents/.

15. Depannage et questions frequentes

Probleme	Cause / solution
L'executable ne s'ouvre pas / crashe silencieusement	Lancer depuis un terminal pour voir l'erreur, ou lire la boite « Erreur inattendue » du <code>_excepthook</code> .
L'API FastAPI n'est pas lancee	L'app affiche « Mode Local » dans la barre d'etat et fonctionne quand meme sur la base locale.
Les chiffres du dashboard semblent differents	Sans API, les indicateurs viennent de la base locale ; avec API, ils viennent du serveur. C'est le comportement attendu.
« Module not found » en build	hiddenimports necessaires : <code>PyQt5.uic</code> , <code>PyQt5.uic.plugins</code> , <code>PyQt5.QtSql</code> .
Texte flou sur Windows	Verifier que le manifeste est embarque et que <code>_setup_high_dpi</code> est execute avant <code>QApplication</code> .

Probleme	Cause / solution
Boutons / champs geants	Ne jamais forcer <code>QSizePolicy.Expanding</code> sur les enfants d'une page (voir <code>_make_responsive</code>).
Base ecrasee / perdue	<code>data/ecole.db</code> vit a cote de l'exe. Sauvegarder ce fichier.
Fichier CSV illisible dans Excel	Le CSV utilise ; et un BOM (<code>utf-8-sig</code>) — compatible Excel francophone.
SmartScreen bloque l'exe Windows	L'exe n'est pas signe : « Informations supplementaires -> Executer quand meme ».
Documents HTML generes	Ils s'ouvrent dans le navigateur par default ; ranges dans <code>data/documents/</code> .

16. Historique des versions

Version	Contenu
1.0.x	Prototype : API FastAPI + prototype web.
1.1.x	Bascule desktop PyQt5 + SQLite, architecture a 3 couches.
1.2.x	Version actuelle : pages metier completes, dashboards par role, client API FastAPI avec repli local (librairie <code>httpx</code>), filtre des eleves par classe par default, certificat filtrable, exe Windows construit via GitHub Actions, exe Linux one-file (<code>build.spec</code>). Restructuration Offline-First : <code>core/</code>, <code>api/</code> (client + <code>sync_worker</code>), <code>repositories/</code> par domaine, <code>database/</code> avec file d'attente <code>file_attente_synchro</code>, <code>views/</code> renomme <code>ui/</code>, barre d'etat « En Ligne » / « Mode Local ». Code nettoye : tous les docstrings supprimees, commentaires courts en francais.

Fin de la documentation. Les references de lignes correspondent au code source de la version 1.2.0 (branche exe).